Lecture 3b:

Decision Trees → Random Forests

Recursive partitioning, ensembles, and feature importance

Omkar Bait (CosmicAI Fellow)
NRAO, Charlottesville

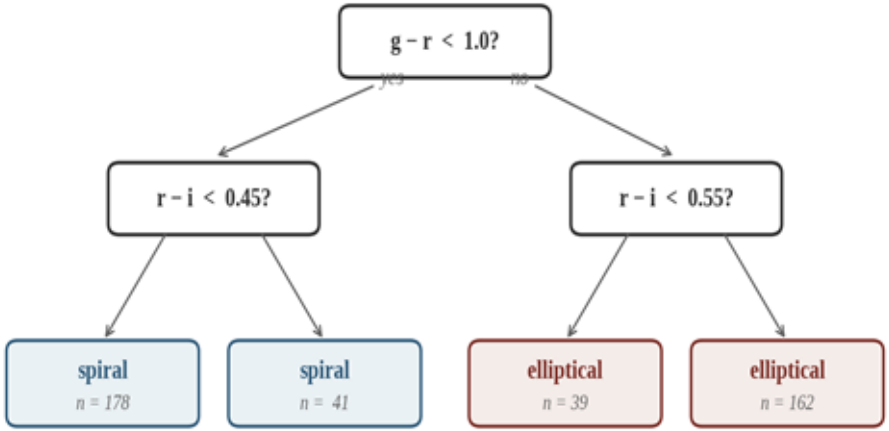
CosmicAI Summer School

What is a Decision Tree?

A series of yes / no questions that route an input to a prediction.

Each question splits the feature space *on a single feature;* every example follows **exactly one** path from root to leaf.

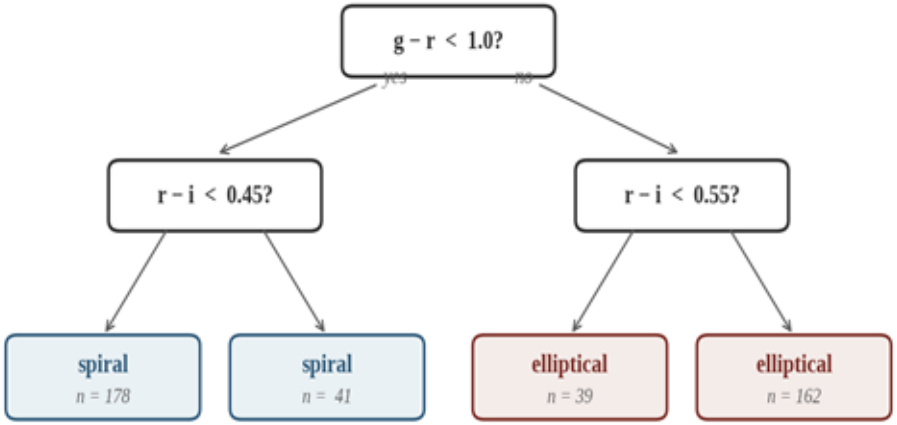
Leaves carry the prediction: *a class label, a probability, or a number.*



A small tree classifying galaxies from two colors

Trees are the easiest model to *draw on a whiteboard* and explain to a non-expert.

Anatomy of a Tree



A small tree classifying galaxies from two colors

Vocabulary

Root node — the topmost question.
Internal node — an interior question.
Leaf (terminal node) — a prediction.
Branch / edge — the yes / no answer.
Depth — longest root-to-leaf path.

Prediction rule: route the example down the tree → read off the leaf value. *Inference is O(depth) — extremely fast.*

Two Flavors — Classification & Regression

Classification tree

y is categorical

- Leaf prediction = most frequent class in that region (majority vote).
- Also outputs class proportions → probability-like score for each class.
- Example: spiral / elliptical / irregular from ugriz colors.

Regression tree

y is continuous

- Leaf prediction = mean (or median) of y in that region.
- Splits chosen to minimize squared-error within each region.
- Example: photometric redshift z from ugriz magnitudes.

Same algorithm, different leaf values. We start with the regression case because the objective is simpler to write down.

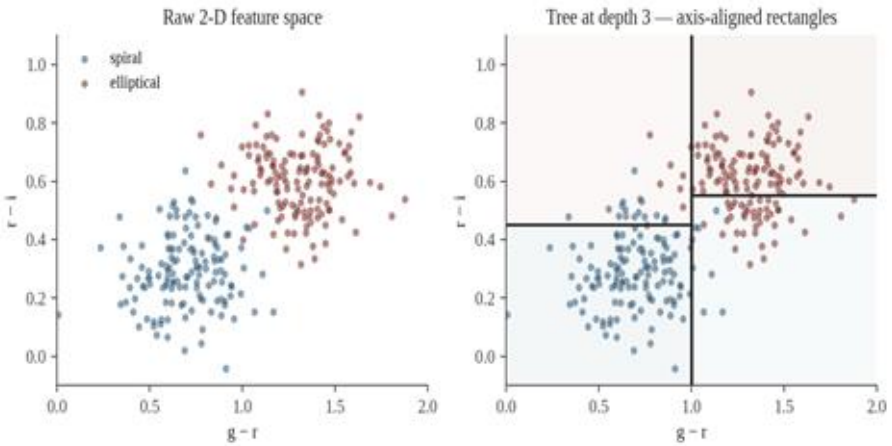
Regression Trees — Partitioning Feature Space

1. Partition the feature space into ***J*** distinct, non-overlapping regions R1, R2, ..., R_J.

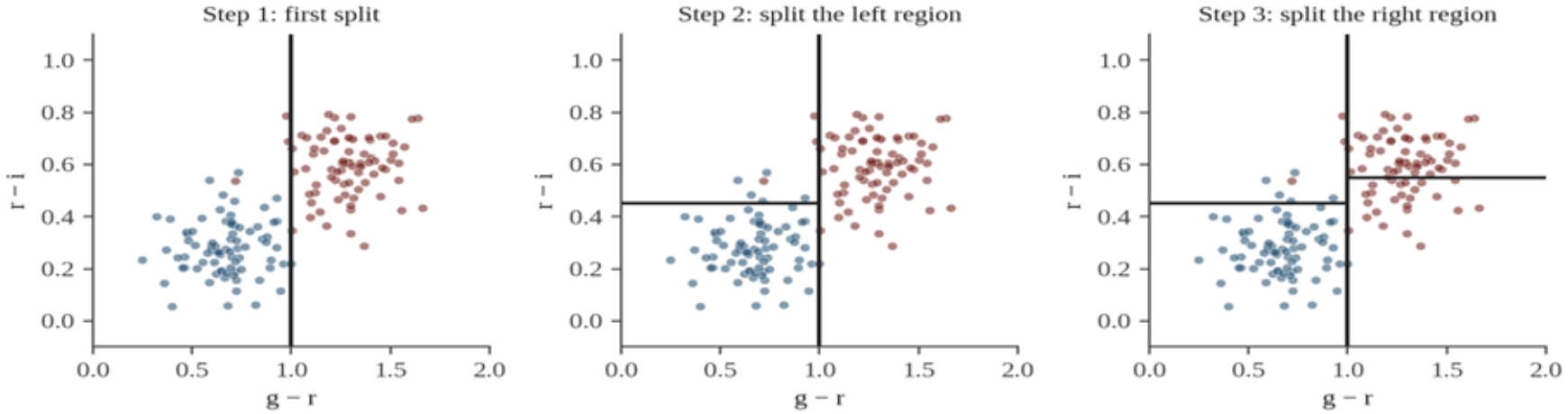
2. In each region, predict the **mean** of the training responses that fall there.

3. For simplicity choose **rectangular** regions (axis-aligned splits).

4. Pick the splits to **minimize mean-square-error** inside every region.



Recursive Binary Splitting



Top-down

Start at the root with all data, then split into smaller regions step by step.

Recursive

Each split creates two children, and the same procedure runs on each child.

Greedy

Pick the locally best split at every step. Don't search ahead — practical, but suboptimal.

Splitting Criteria — Gini & Entropy

For classification, MSE no longer makes sense. We need a measure of node impurity.

Gini impurity

$$G = 1 - \Sigma_k \; p_k^2$$

p_k = proportion of class k in the node. Default in sklearn.

Entropy

$$H = - \Sigma_k \; p_k \; \log p_k$$

Roots in information theory.

Split quality: minimize the weighted-average impurity of the two child nodes. In practice ***Gini ≈ Entropy*** — either works.

Pruning — Why?

A large tree can fit ***any*** training set perfectly:

just keep splitting until every leaf has one example.

But the model has memorized noise:

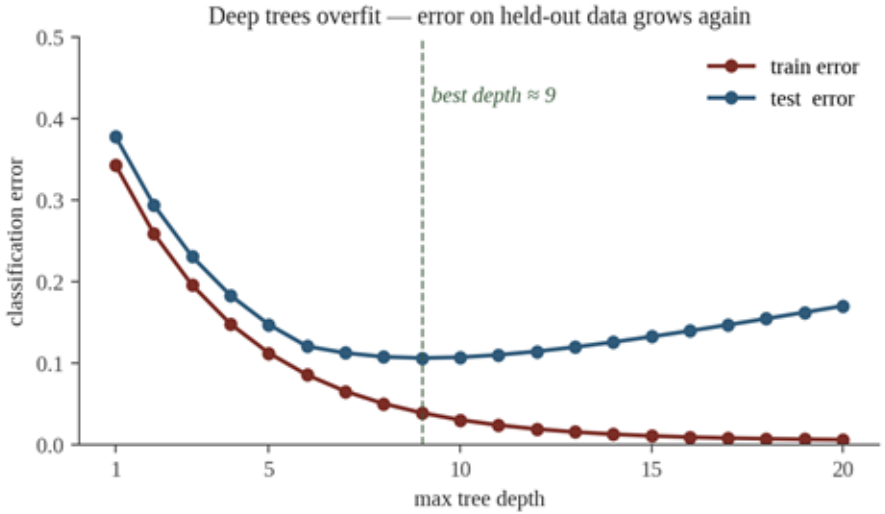
train error → 0, test error climbs.

Recall Lec 3a: the test set is the only honest judge.

Two cures:

Pre-pruning: stop splitting when the tree gets deep enough.

Post-pruning: grow fully, then chop branches that don't help on validation.



Pre-pruning hyperparameters in sklearn:

`max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes.`

Pruning — Cost-Complexity (post-pruning)

Grow the tree to full depth, then remove the weakest branches.

$$\text{minimize} \quad \sum_{m=1}^{|T|} \sum_{i:\, x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

What the symbols mean:

- `|T|` total number of leaves in the (sub-)tree.
- `α` hyperparameter trading off *fit* vs *size*.

How to pick α:

- `α = 0:` no penalty → full tree.
- `α large:` tiny tree, even just the root.
- **Sweet spot:** minimize error on the validation set.

sklearn: `DecisionTreeClassifier(ccp_alpha=α)` — `scan α with cross-validation.`

Predicting from a Classification Tree

Once trained, prediction is mechanical:

Route the new example down the tree until it lands in a leaf.

Predict the most frequent class (majority vote) among the training examples in that leaf.

Also report class proportions in the leaf as a probability-like score:

```
P^(spiral | x) ≈ n_spiral / n_total in leaf
```

Worked example

A new galaxy with $g - r = 0.9$, $r - i = 0.30$:

1. $g - r < 1.0$? → yes, go left.

2. $r - i < 0.45$? → yes, go left.

3. Leaf has n = 178 examples:

```
170 spiral,  6 elliptical,  2 irregular.
```

Prediction: *spiral* `with P^ = 170 / 178 = 0.96`

Strengths & Weaknesses of Single Trees

Strengths

Intuitive: easy to draw, explain, modify.

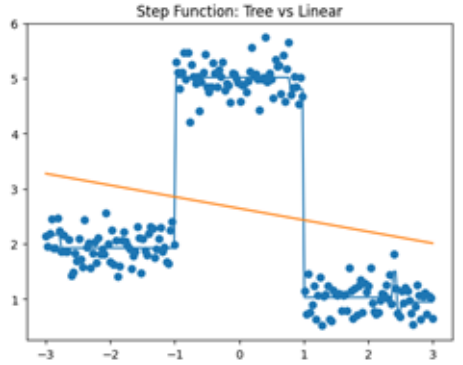
No scaling needed: trees are invariant to transforms.

Mixed feature types: numerical and categorical handled together.

Captures non-linear, non-monotonic structure (the partition).

Built-in feature importance.

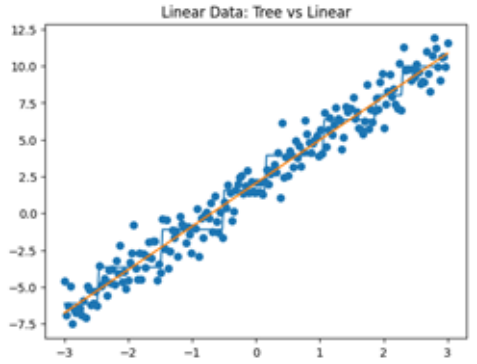
Fast inference O(depth).



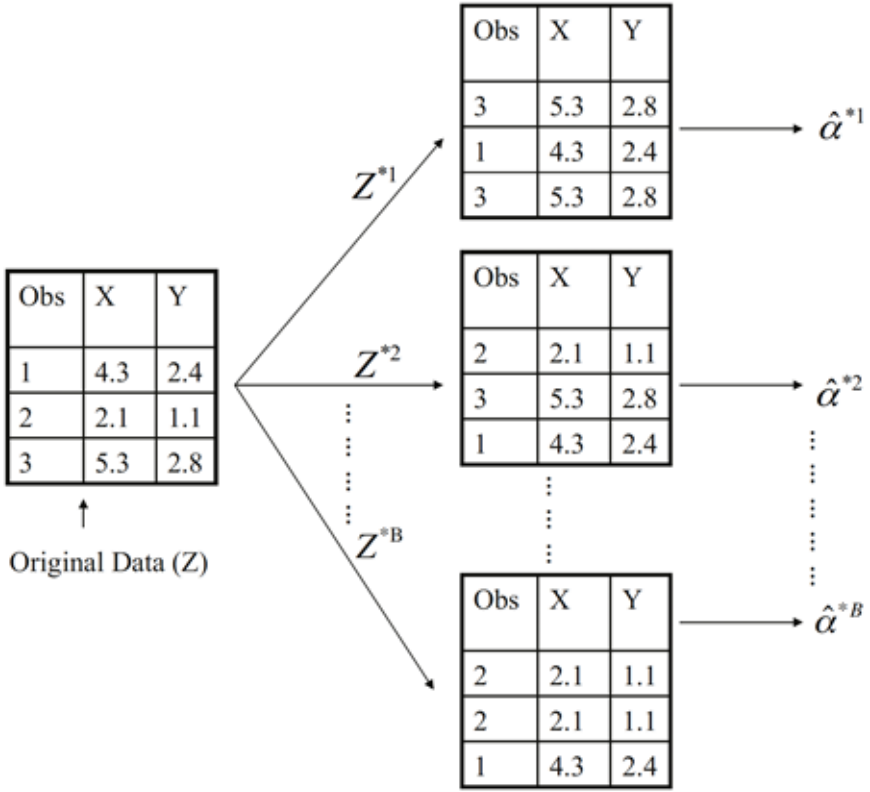
Weaknesses

High variance: small data change → very different tree.

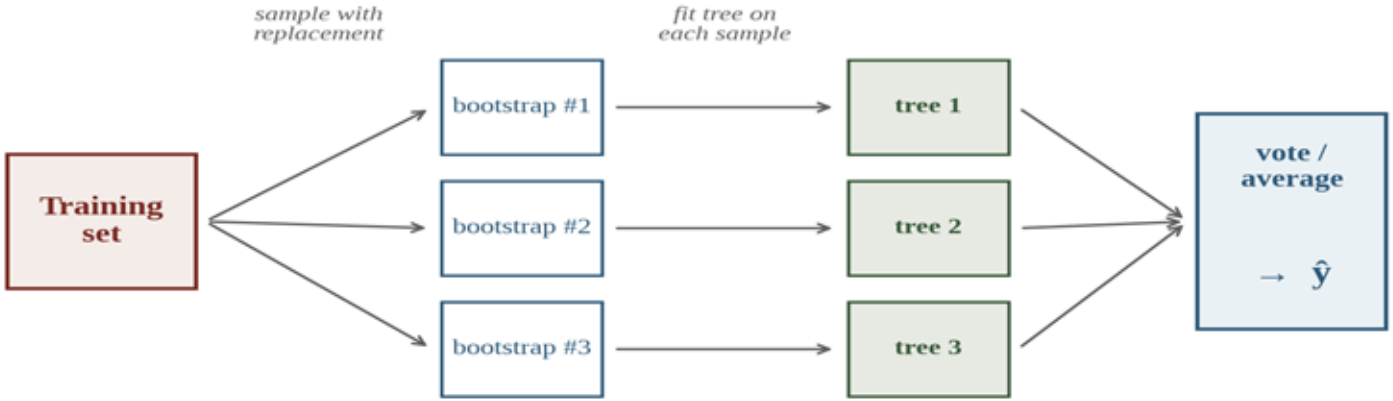
Trees can underperform on some simple linear tasks.
Very easy to overfit.



Bootstrap


Obs X Y
1 4.3 2.4
2 2.1 1.1
3 5.3 2.8
Original Data (Z)
Z^{*1}
Z^{*2}
Z^{*B}
Obs X Y
3 5.3 2.8
1 4.3 2.4
3 5.3 2.8
Obs X Y
2 2.1 1.1
3 5.3 2.8
1 4.3 2.4
Obs X Y
2 2.1 1.1
2 2.1 1.1
1 4.3 2.4
$\hat{\alpha}^{*1}$
$\hat{\alpha}^{*2}$
$\hat{\alpha}^{*B}$

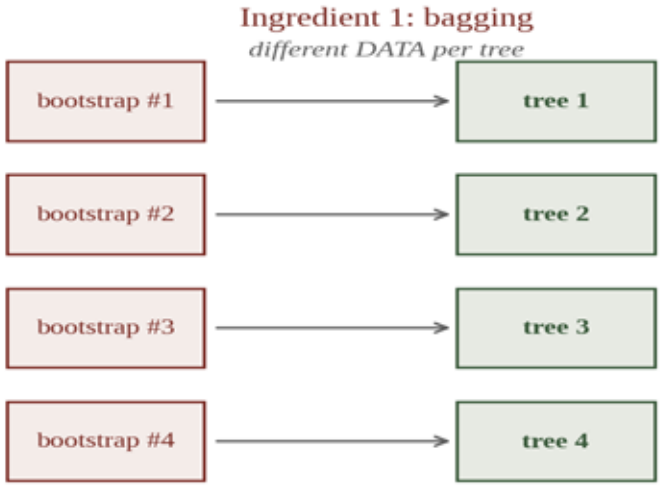
From One Tree to Many — Bagging

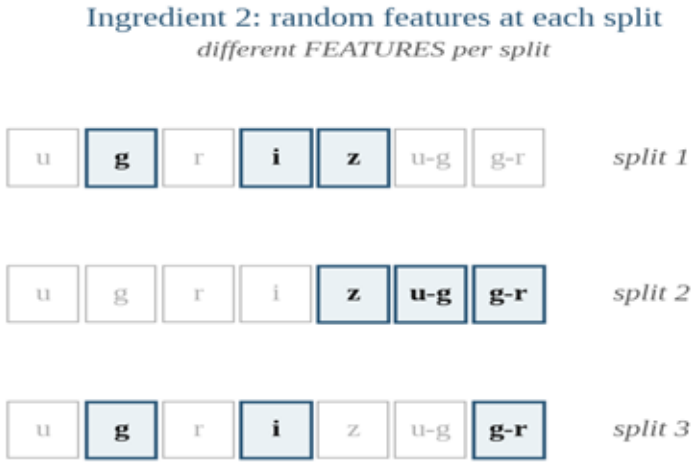


Bagging = Bootstrap + Aggregating.
1. Draw B bootstrap samples *(with replacement)*.
2. Train one tree per sample.
3. Aggregate: mean for regression, vote for classification.

Why this helps:
Each tree is trained on a different bootstrap, so averaging cancels the noise (variance ↓).
Bias stays the same (trees are fully grown).

Random Forests — Two Ingredients

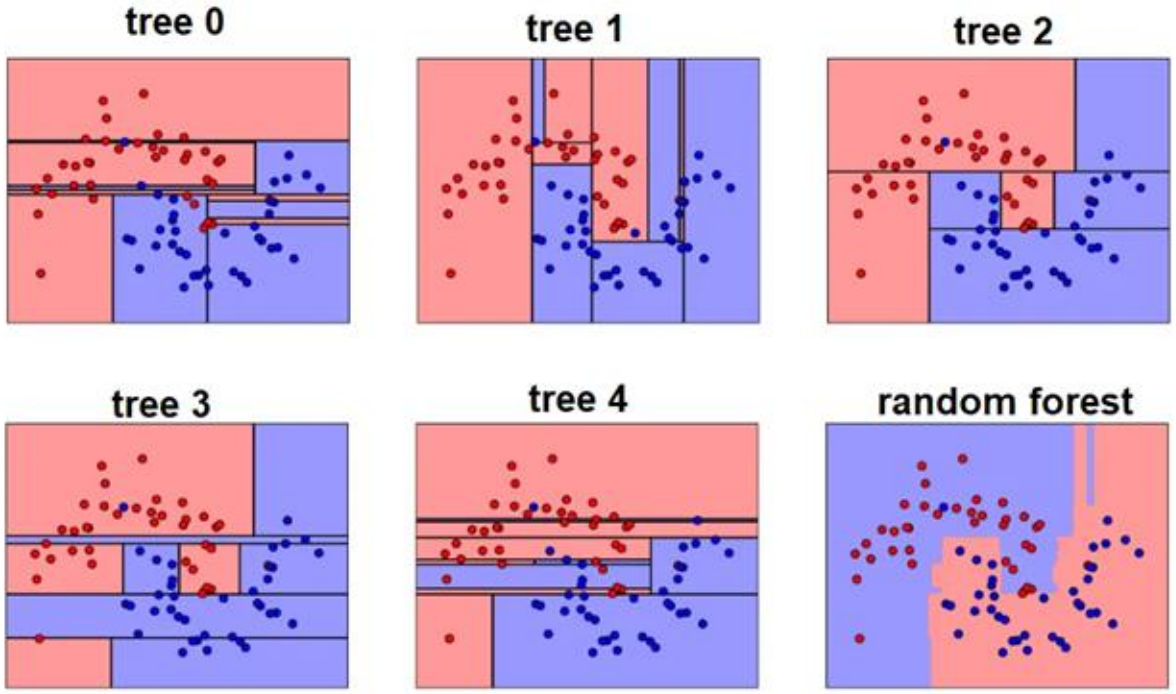




1. Bagging (data randomness):
different bootstrap sample per tree → different data.

2. Feature subsampling (feature randomness):
at every split, only a random subset of features is considered.

Random Forest on Spiral Data



Quick Check 1

QUIZ

What single property distinguishes a Random Forest from plain bagged decision trees?

A RF uses many trees; bagging uses just one

B RF uses bootstrap sampling; bagging does not

C RF also samples FEATURES at each split

D RF is more interpretable than bagged trees

Pick A, B, C, or D — then advance.

Quick Check 1 — Answer

ANSWER

A Many trees vs one

Both use many trees — the difference is structural.

B Bootstrap vs not

Both use bootstrap sampling (bagging).

C **RF samples FEATURES too** ✓

At each split, only √d features are considered.

D RF more interpretable

Forests are less interpretable than a single tree.

Why this matters: feature subsampling forces different trees to use different features → ***the trees become decorrelated.*** Averaging decorrelated noisy predictions is far more powerful than averaging correlated ones.

Aggregating Predictions & OOB Score

Aggregation

Regression: average the B tree predictions.

$\hat{y} = (1/B) \cdot \Sigma_b \ \hat{y}_b(x)$

Classification: majority vote, or average the class probabilities and then argmax.

Why it works: variance of the mean = variance of one / B (if the trees were independent).

Out-Of-Bag (OOB) score

A free held-out estimate, no separate validation set needed.

For each training example x_i:
average the predictions from only those trees that DID NOT see x_i in their bootstrap.

sklearn:

```
RandomForestClassifier(oob_score=True).oob
_score_
```

Feature Importance

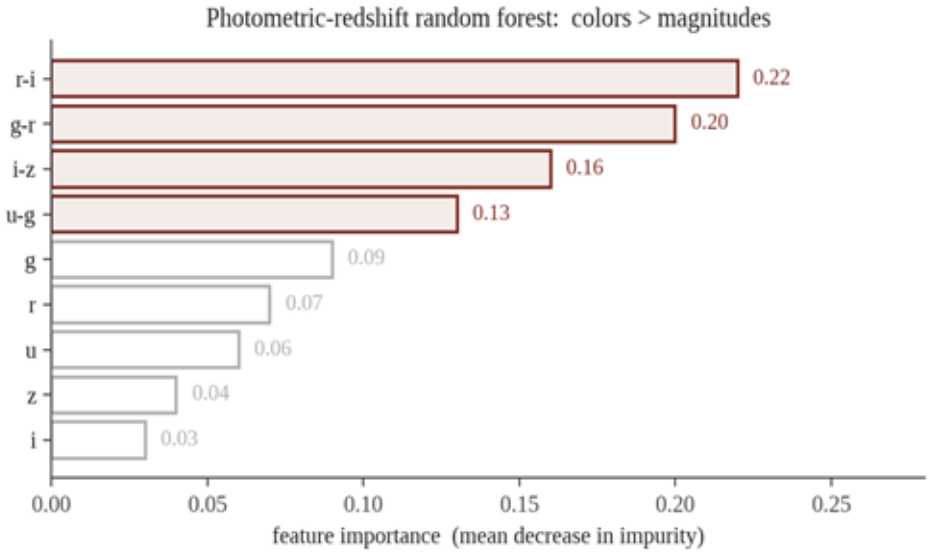
How important is each feature? Two common definitions.

Mean Decrease in Impurity (MDI):

at each split, record the impurity drop times the number of samples; sum across all splits and all trees that use that feature; normalize.

Permutation importance:

for each feature, shuffle its values across rows and measure how much the OOB / test score drops.



Code — Random Forest in sklearn

Plugs straight into the Pipeline / CV scaffolding from Lec 2 & 3a.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500,          # how many trees
    max_depth=None,           # fully grown (forest will average out)
    max_features="sqrt",       # max features set to sqrt(n_features)
    min_samples_leaf=5,          # lightweight pre-pruning
    oob_score=True,           # free held-out estimate
    n_jobs=-1, random_state=42,
)

rf.fit(X_train, y_train)
print("OOB score:", rf.oob_score_)

# Feature importance ranking
import pandas as pd
pd.Series(rf.feature_importances_, index=X_train.columns).sort_values()
```

Recap — Trees and Forests

1	**Decision tree**	Recursive binary splitting of feature space; predict per leaf.
2	**Splitting criteria**	MSE for regression; Gini or Entropy for classification.
3	**Pruning**	Pre-pruning (max_depth, min_samples_leaf) or cost-complexity post-pruning.
4	**Bagging**	Average B bootstrap-trained trees → drops variance.
5	**Random Forest**	Bagging + feature subsampling at each split → decorrelated trees.
6	**Free goodies**	OOB score, feature importance, fast inference.

Next → Lecture 4: putting it all together — photometric-redshift case study with scikit-learn.